

ДЗ к лекции 1-2

Представление данных

Задание 1. Установка Rust и rustup

Цель: установить Rust через rustup, проверить, что инструменты доступны из терминала, и понять, какие команды понадобятся в курсе.

Что нужно сделать

Установите Rust с помощью официального установщика rustup.

После установки проверьте, что доступны команды:

```
rustc --version  
cargo --version  
rustup --version
```

Каждая команда должна вывести версию установленного инструмента.

Что такое rustup

rustup — это менеджер версий Rust.

Он позволяет:

- установить Rust;
- обновлять компилятор;
- переключаться между версиями Rust;
- устанавливать дополнительные компоненты;
- управлять toolchain'ами.

Что такое rustc

rustc — это компилятор Rust.

Он превращает файл с кодом Rust в исполняемую программу.

Что такое cargo

cargo — это главный инструмент Rust-разработчика.

Он умеет:

- создавать проекты;
- собирать код;
- запускать программы;
- запускать тесты;
- подключать зависимости.

Если все три команды работают, задание выполнено.

Задание 2. Hello, World!

Цель: создать и скомпилировать первую программу на Rust.

Напишите программу, которая выводит: Hello, World!

Вариант 1. Через один файл

1. Создайте файл `main.rs`
2. Скомпилируйте программу: `rustc main.rs`
3. Запустите её: `./main`
4. Ожидаемый вывод: `Hello, World!`

Вариант 2. Через Cargo

1. Создайте новый проект: `cargo new hello`
2. Перейдите в папку проекта: `cd hello`
3. Запустите программу: `cargo run`
4. Ожидаемый вывод: `Hello, World!`

Можно запустить без служебного вывода: `cargo run --quiet`

Задание 3. Count bytes

Цель: познакомиться с базовым вводом/выводом в Rust.

Напишите программу `ba01`, которая читает данные из **standard input** и печатает в **standard output** количество прочитанных байтов.

Примеры запуска

```
$ cargo run
hello
5
```

```
$ echo foo | cargo run --quiet
4
```

Обратите внимание: `echo foo` передаёт не только символы `f`, `o`, `o`, но и перевод строки `\n`, поэтому результат равен 4.

```
$ cargo run --quiet < /dev/null
0
```

```
$ yes | head -c 1000 | cargo run --quiet
1000
```

```
$ ./hello | cargo run --quiet
13
```

```
$ echo 🦀 | cargo run --quiet
5
```

Почему 5? Эмодзи `crab` в UTF-8 занимает 4 байта, а `echo` добавляет перевод строки `\n`, который занимает ещё 1 байт.

Требования

Программа должна:

- читать все данные из `stdin` до EOF;
- считать именно байты, а не Unicode-символы;
- вывести одно число — количество байтов;

- после числа вывести перевод строки;
- корректно работать с пустым вводом;
- корректно работать с бинарными данными.

Подсказка

В Rust стандартный ввод можно получить через:

```
std::io::stdin()
```

Стандартный ввод можно читать как поток байтов. Для этого пригодятся типы и трейты из модуля:

```
use std::io::{self, Read};
```

Задание 4. Count words and lines

Цель: научиться внимательно читать спецификацию для программы ввода/вывода и аккуратно обрабатывать разные случаи: пустой ввод, пробелы, переводы строк, слова и байты.

Напишите программу `ba02`, которая читает данные из standard input и печатает три числа:

- количество строк
- количество слов
- количество байтов

Именно в таком порядке:

```
lines words bytes
```

Что считается строкой

Строка — это последовательность из нуля или более байтов, которая заканчивается символом перевода строки `\n`.

То есть количество строк — это количество байтов `\n` во входных данных.

Что считается словом

Слово — это последовательность из одного или более **непробельных** символов.

Пробельными считаются, например:

- пробел
- табуляция
- перевод строки `\n`

Несколько пробелов подряд не создают пустые слова.
Например: "hello rust" содержит **2 слова**, а не 4.

Примеры запуска

```
$ echo foo | cargo run --quiet
1 1 4
```

Почему 1 1 4:

- 1 строка — `echo` добавляет `\n`
- 1 слово — `foo`
- 4 байта — `f`, `o`, `o`, `\n`

```
$ cargo run --quiet < /dev/null
0 0 0
```

```
$ yes | head -c 1000 | cargo run --quiet
500 500 1000
```

Почему 500 500 1000:

- `yes` печатает `y\n`
- каждая запись занимает 2 байта
- 1000 байт = 500 повторений `y\n`

```
$ yes Hello | head -c 1000 | cargo run --quiet
166 167 1000
```

```
$ ./hello | cargo run --quiet
1 2 13
```

```
$ echo " hi " | cargo run --quiet
1 1 7
```

Почему 1 1 7:

- строка: одна, потому что echo добавляет \n
- слово: одно, hi
- байты: два пробела + h + i + два пробела + \n = 7

Требования

Программа должна:

- читать все данные из stdin до EOF;
- вывести три числа: lines words bytes;
- считать строки как количество символов \n;
- считать слова как последовательности непробельных символов;
- считать именно **байты**, а не Unicode-символы;
- корректно работать с пустым вводом;
- корректно работать с несколькими пробелами подряд;
- корректно работать с табами и переводами строк;
- корректно работать с бинарными данными.

Подсказка

В Rust стандартный ввод можно читать как поток байтов:

```
use std::io::{self, Read};
```

Для подсчета слов удобно хранить состояние: сейчас мы внутри слова или нет?

Например:

если текущий байт — пробельный:

мы больше не внутри слова

если текущий байт — непробельный:

если раньше мы были не внутри слова:

началось новое слово

теперь мы внутри слова

Для проверки пробельных ASCII-байтов можно использовать метод:
`byte.is_ascii_whitespace()`

Важные тестовые случаи

Обязательно проверьте программу на таких входах:

```
$ printf "" | cargo run --quiet
0 0 0
```

```
$ printf "hello" | cargo run --quiet
0 1 5
```

```
$ printf "hello\n" | cargo run --quiet
1 1 6
```

```
$ printf "hello rust\n" | cargo run --quiet
1 2 11
```

```
$ printf " hello rust \n" | cargo run --quiet
1 2 17
```

```
$ printf "a\tb\nc" | cargo run --quiet
1 3 5
```

Задание 5. Sort arguments

Цель: научиться работать с аргументами командной строки в Rust, сортировать данные и понимать, что аргументы программы — это тоже строки, переданные операционной системой при запуске.

Напишите программу `ba03`, которая печатает свои аргументы командной строки в **лексикографическом порядке**, по одному аргументу на строку.

Имя самой программы учитывать не нужно.

Что такое лексикографический порядок

Лексикографический порядок — это “словарный” порядок строк.

Например: $a < b < c$

Но важно помнить: сравнение строк в Rust по умолчанию чувствительно к регистру.

Поэтому заглавные буквы идут раньше строчных: $A < a$

Примеры запуска

```
$ cargo run --quiet -- a b c  
a  
b  
c
```

```
$ cargo run --quiet -- e d c b a  
a  
b  
c  
d  
e
```

```
$ cargo run --quiet -- A a A a A a  
A  
A  
A  
a  
a  
a
```

```
$ cargo run --quiet --
```

Если аргументов нет, программа ничего не печатает.

```
$ cargo run --quiet -- hello, world, this is a  
program  
a  
hello,  
is  
program
```



```
this  
world,
```

Обратите внимание: `hello`, и `world`, включают запятую, потому что для `shell` это часть аргумента.

Зачем нужен `--`

В команде: `cargo run --quiet -- hello world`
двойной дефис `--` отделяет аргументы Cargo от аргументов вашей программы.

То есть: `cargo run --quiet`
это параметры для Cargo, а всё после `-- hello world` передаётся уже вашей программе.

Требования

Программа должна:

- прочитать аргументы командной строки;
- не учитывать имя самой программы;
- сортировать аргументы в лексикографическом порядке;
- вывести каждый аргумент на отдельной строке;
- корректно работать, если аргументов нет;
- корректно работать с повторяющимися аргументами;
- не удалять запятые, точки и другие символы из аргументов.

Подсказка

В Rust аргументы командной строки можно получить через:

```
std::env::args()
```

Например:

```
fn main() {  
    for arg in std::env::args() {  
        println!("{}", arg);  
    }  
}
```

Но первый аргумент — это обычно имя программы. Его нужно пропустить. Для этого можно использовать `.skip(1)`:

```
fn main() {  
    for arg in std::env::args().skip(1) {  
        println!("{}", arg);  
    }  
}
```

Версия 1: использовать стандартную сортировку

Версия 2: реализовать сортировку самостоятельно

Более сложный вариант: реализуйте сортировку вручную. Например, можно использовать **insertion sort**.

Важно! Напишите тесты на функцию

Что важно понять

Это задание тренирует сразу несколько вещей:

- аргументы командной строки отличаются от `stdin`;
- первый аргумент обычно является именем программы;
- строки можно сравнивать лексикографически;
- сортировка — это не магия, а алгоритм перестановки элементов;
- `Vec<String>` — удобная структура для хранения динамического списка строк;
- стандартная библиотека Rust часто уже содержит готовые инструменты, но полезно понимать, как это работает.

Главное отличие от предыдущих заданий:

- `Count bytes / Count words` читали данные из `stdin`.
- `Sort arguments` читает данные из `command-line arguments`.

То есть, это разные способы передать данные программе:

- `echo hello | cargo run --quiet`
- `cargo run --quiet -- hello`

Задание 6. Unsigned integers и overflow

Цель: понять, как целые числа без знака хранятся в фиксированном количестве битов, почему у `u8` диапазон `0..=255`, и что происходит, когда результат вычисления не помещается в тип.

Напишите небольшую библиотеку с функциями для сложения `u8` в трёх разных режимах обработки переполнения:

- `checked`
- `wrapping`
- `saturating`

Реализуйте функции

```
pub fn add_u8_checked(a: u8, b: u8) -> Option<u8> {  
    todo!()  
}
```

```
pub fn add_u8_wrapping(a: u8, b: u8) -> u8 {  
    todo!()  
}
```

```
pub fn add_u8_saturating(a: u8, b: u8) -> u8 {  
    todo!()  
}
```

Примеры поведения

`checked_add`

- Если переполнения нет, возвращает `Some(result)`.
- Если переполнение есть, возвращает `None`.

```
assert_eq!(add_u8_checked(10, 20), Some(30));  
assert_eq!(add_u8_checked(255, 1), None);
```

wrapping_add

- При переполнении “заворачивается” по кругу.

```
assert_eq!(add_u8_wrapping(10, 20), 30);
assert_eq!(add_u8_wrapping(255, 1), 0);
```

saturating_add

- При переполнении возвращает максимально возможное значение. Для u8 это 255.

```
assert_eq!(add_u8_saturating(10, 20), 30);
assert_eq!(add_u8_saturating(255, 1), 255);
```

Тесты

```
#[test]
fn unsigned_overflow_modes() {
    assert_eq!(add_u8_checked(255, 1), None);
    assert_eq!(add_u8_wrapping(255, 1), 0);
    assert_eq!(add_u8_saturating(255, 1), 255);

    assert_eq!(add_u8_checked(10, 20), Some(30));
    assert_eq!(add_u8_wrapping(10, 20), 30);
    assert_eq!(add_u8_saturating(10, 20), 30);
}
```

Программа должна:

- реализовать функции `add_u8_checked`, `add_u8_wrapping`, `add_u8_saturating`;
- корректно обрабатывать сложение без переполнения;
- корректно обрабатывать сложение с переполнением;
- не использовать `panic!` для обработки `overflow`;
- проходить все тесты;

- Нельзя использовать встроенные: `checked_add`, `wrapping_add`, `saturating_add`. Нужно реализовать поведение вручную.

Bonus. Hello pixels

Цель: понять, что изображение тоже можно представить как набор битов, а один байт может быть не только числом, но и строкой из 8 пикселей.

В этой задаче вы будете работать с маленькой картинкой размером **8×8**

Каждый пиксель кодируется одним битом:

- `#` = 1
- `.` = 0

Дана картинка:

```
..####..  
.#....#.  
#.#...#.#  
#..##...#  
#.....#  
#.#...#.#  
.#....#.  
..####..
```

Каждая строка картинки содержит **8 пикселей**, значит, каждую строку можно представить как один **u8**.

Например: `..####..`

превращается в биты: `00111100`

А это число: 0b0011_1100

или в HEX: 0x3C

Что нужно сделать

Реализуйте три функции:

```
pub fn parse_bitmap_8x8(lines: [&str; 8]) -> [u8; 8] {
    todo!()
}

pub fn render_bitmap_8x8(bytes: [u8; 8]) -> [String; 8] {
    todo!()
}

pub fn invert_bitmap_8x8(bytes: [u8; 8]) -> [u8; 8] {
    todo!()
}
```

Часть 1. Parse bitmap

Функция `parse_bitmap_8x8` должна принять картинку как массив из 8 строк и вернуть массив из 8 байтов.

Каждая строка превращается в один `u8`.

Правила:

- `#` становится битом 1
- `.` становится битом 0

Пример:

```
let image = [  
  "..####..",  
  ".#....#.",  
  "#.#..#.#",  
  "#..##..#",  
  "#.....#",  
  "#.#..#.#",  
  ".#....#.",  
  "..####..",  
];  
  
let bytes = parse_bitmap_8x8(image);
```

Ожидаемый результат:

```
[  
  0b0011_1100,  
  0b0100_0010,  
  0b1010_0101,  
  0b1001_1001,  
  0b1000_0001,  
  0b1010_0101,  
  0b0100_0010,  
  0b0011_1100,  
]
```

Часть 2. Render bitmap

Функция `render_bitmap_8x8` должна выполнить обратную операцию.

Она принимает массив байтов:

```
[  
  0b0011_1100,  
  0b0100_0010,
```

```
    0b1010_0101,  
    0b1001_1001,  
    0b1000_0001,  
    0b1010_0101,  
    0b0100_0010,  
    0b0011_1100,  
]
```

И возвращает картинку:

```
..####..  
.#....#.  
#.#..#.#  
#..##..#  
#.....#  
#.#..#.#  
.#....#.  
..####..
```

Часть 3. Invert bitmap

Функция `invert_bitmap_8x8` должна инвертировать картинку:

- 0 становится 1
- 1 становится 0

То есть:

- . становится #
- # становится .

Если исходная строка: ..####..

то после инверсии получится: ##....##

Для исходной картинки:

```
..####..  
.#....#.  
#.#...#.#  
#...##...#  
#.....#  
#.#...#.#  
.#....#.  
..####..
```

инвертированная картинка должна быть:

```
##.....##  
#.####.#  
.#.##.##.  
.##...##.  
.#####.  
.#.##.##.  
#.####.#  
##.....##
```

Примеры запуска

Можно сделать небольшой `main`, чтобы посмотреть результат:

```
fn main() {  
    let image = [  
        "..####..  
        ".#....#..  
        "#.#...#..  
        "#...##...#  
        "#.....#  
        "#.#...#..  
        ".#....#..  
        "..####..  
    ];  
  
    let bytes = parse_bitmap_8x8(image);
```

```

println!("Bytes:");
for byte in bytes {
    println!("{byte:08b}  0x{byte:02X}");
}

println!();
println!("Rendered:");

for line in render_bitmap_8x8(bytes) {
    println!("{line}");
}

println!();
println!("Inverted:");

for line in render_bitmap_8x8(invert_bitmap_8x8(bytes)) {
    println!("{line}");
}
}

```

Ожидаемый вывод:

```

Bytes:
00111100  0x3C
01000010  0x42
10100101  0xA5
10011001  0x99
10000001  0x81
10100101  0xA5
01000010  0x42
00111100  0x3C

```

Rendered:

```
..####..  
.#....#.  
#.#...#.#  
#...##...#  
#.....#  
#.#...#.#  
.#....#.  
..####..
```

Inverted:

```
##....##  
#.####.#  
.#.##.##.  
.##...##.  
.#####.  
.#.##.##.  
#.####.#  
##....##
```

Тесты

Добавьте тесты на все функции

Требования

Программа должна:

- работать с картинкой ровно **8×8**;
 - представлять каждую строку как один `u8`;
 - считать `#` битом `1`;
 - считать `.` битом `0`;
 - уметь восстановить картинку из массива `[u8; 8]`;
 - уметь инвертировать картинку;
 - проходить все тесты.
-

Подсказка 1. Как выставить бит

Чтобы выставить конкретный бит, можно использовать побитовое OR.

Подсказка 2. Как связать позицию символа и бит

В строке 8 символов:

`..####..`

Позиции идут слева направо:

0	1	2	3	4	5	6	7
.	.	#	#	#	#	.	.

Но в байте старший бит находится слева:

бит:	7	6	5	4	3	2	1	0
	0	0	1	1	1	1	0	0

Значит, символ с индексом `i` соответствует биту:

`7 - i`

Например:

```
i = 0 -> bit 7
i = 1 -> bit 6
i = 2 -> bit 5
...
i = 7 -> bit 0
```

Подсказка 3. Как проверить бит

Чтобы проверить, установлен ли бит, можно использовать маску:

```
let is_set = byte & (1 << bit_index) != 0;
```

Если бит установлен, печатаем #

Если бит не установлен, печатаем .

Подсказка 4. Как инвертировать байт

Для инверсии всех битов в байте можно использовать !:

```
let inverted = !byte;
```

Так как картинка занимает ровно 8 бит в каждой строке, это инвертирует все пиксели строки.

Что важно понять

Это задание выглядит как работа с картинкой, но на самом деле это работа с представлением данных.